

Automated Parking Valet System: Design, Implementation, and Simulation

Priyansh Lathigara - 202201449

December 5, 2024

Contents

1	Introduction	3
1.1	Objectives	3
2	System Model	3
2.1	Kinematic Bicycle Model	3
3	State-Space Modeling	4
3.1	State Variables	4
3.2	Inputs	4
3.3	System Dynamics	4
3.4	Linearized State-Space Form	4
3.5	State Matrices	5
4	Control Strategies	5
4.1	Lateral Control	5
4.2	Longitudinal Control	5
4.2.1	Proportional-Integral (PI) Controller	5
4.3	Control Objective	5
4.4	Controller Design	6
4.5	Implementation Steps	6
4.6	Discrete-Time Implementation	6
4.7	Tuning the Controller	6
4.8	Application in Longitudinal Control	7
4.9	Performance Evaluation	7
5	Simulation Setup	7
5.1	Simulator Design	7
5.2	Path Planning	7
6	Results and Analysis	8
6.1	Trajectory Tracking	8
6.2	Parking Maneuvers	8
6.3	Error Analysis	8

7	Related Research	9
7.1	Influential Papers	9
7.2	Application of Research	9
8	Conclusion and Future Work	9
A	Appendix	10

1 Introduction

The Automated Parking Valet system enables vehicles to autonomously navigate and park in designated spaces. This report details the system's design, control strategies, and simulation results. Leveraging a kinematic bicycle model, feedback controllers, and advanced path planning techniques, the system ensures efficient and accurate parking maneuvers.

1.1 Objectives

- Develop a kinematic-based control system for trajectory following.
- Simulate realistic vehicle dynamics using a bicycle model.
- Ensure robust parking under diverse conditions.

2 System Model

The system employs a kinematic bicycle model for vehicle motion, given its applicability to low-speed scenarios.

2.1 Kinematic Bicycle Model

The kinematic equations governing the vehicle are:

$$\dot{x}_r = v_r \cos(\theta), \quad (1)$$

$$\dot{y}_r = v_r \sin(\theta), \quad (2)$$

$$\dot{\theta} = \frac{v_r}{l} \tan(\delta), \quad (3)$$

$$\dot{v}_r = a_r, \quad (4)$$

where:

- (x_r, y_r, θ) : Vehicle pose in world coordinates.
- v_r : Rear-wheel speed.
- a_r : Rear-wheel acceleration.
- l : Wheelbase.
- δ : Steering angle.

The front-wheel position and speed are derived as:

$$x_f = x_r + l \cos(\theta), \quad (5)$$

$$y_f = y_r + l \sin(\theta), \quad (6)$$

$$v_f = \frac{v_r}{\cos(\delta)}. \quad (7)$$

3 State-Space Modeling

The kinematic bicycle model for the Automated Parking Valet system can be expressed using state-space representation.

3.1 State Variables

The state vector is defined as:

$$\mathbf{x} = \begin{bmatrix} x_r \\ y_r \\ \theta \\ v_r \end{bmatrix},$$

where:

- x_r : Rear axle x-coordinate.
- y_r : Rear axle y-coordinate.
- θ : Heading angle.
- v_r : Longitudinal velocity at the rear wheel.

3.2 Inputs

The input vector is defined as:

$$\mathbf{u} = \begin{bmatrix} \delta \\ a_r \end{bmatrix},$$

where:

- δ : Steering angle.
- a_r : Rear-wheel acceleration.

3.3 System Dynamics

The state-space representation of the system is:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} v_r \cos(\theta) \\ v_r \sin(\theta) \\ \frac{v_r}{l} \tan(\delta) \\ a_r \end{bmatrix}.$$

3.4 Linearized State-Space Form

For control design purposes, the system can be linearized around a nominal trajectory. The linearized state-space form is given by:

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu},$$

$$\mathbf{y} = \mathbf{Cx} + \mathbf{Du},$$

where:

- **A**: State matrix, describing the linearized system dynamics.
- **B**: Input matrix, describing how inputs affect the system states.
- **C**: Output matrix, mapping states to outputs.
- **D**: Feedthrough matrix.

3.5 State Matrices

For the kinematic bicycle model, the state and input matrices near a linearization point can be computed as:

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & -v_r \sin(\theta) & \cos(\theta) \\ 0 & 0 & v_r \cos(\theta) & \sin(\theta) \\ 0 & 0 & 0 & \frac{\tan(\delta)}{l} \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & \frac{v_r}{l \cos^2(\delta)} \\ 0 & 1 \end{bmatrix}.$$

The matrices **C** and **D** depend on the chosen outputs and are often identity and zero matrices, respectively, for this system.

4 Control Strategies

The system employs two primary controllers:

4.1 Lateral Control

The lateral control adjusts the steering angle to minimize lateral errors and align the vehicle with the reference path.

4.2 Longitudinal Control

The longitudinal control uses a Proportional-Integral (PI) approach to compute throttle and braking commands based on velocity error.

4.2.1 Proportional-Integral (PI) Controller

The Proportional-Integral (PI) controller is a fundamental feedback control mechanism used to minimize the error between the desired and actual system outputs. In the Automated Parking Valet system, the PI controller is applied to the longitudinal control to regulate the vehicle's speed.

4.3 Control Objective

The goal of the PI controller is to ensure that the vehicle achieves and maintains a desired velocity v_d by minimizing the velocity error $e_v(t)$, defined as:

$$e_v(t) = v_d(t) - v_r(t),$$

where:

- $v_d(t)$: Desired velocity at time t .
- $v_r(t)$: Actual longitudinal velocity at the rear wheel at time t .

4.4 Controller Design

The control law for the PI controller is expressed as:

$$u(t) = K_p e_v(t) + K_i \int_0^t e_v(\tau) d\tau,$$

where:

- $u(t)$: Control input (acceleration or deceleration) at time t .
- K_p : Proportional gain, determining the response to the current error.
- K_i : Integral gain, correcting cumulative errors over time.
- $\int_0^t e_v(\tau) d\tau$: Integral of the error, addressing steady-state error.

4.5 Implementation Steps

The implementation of the PI controller involves:

1. Calculating the velocity error $e_v(t) = v_d(t) - v_r(t)$.
2. Computing the proportional term $K_p e_v(t)$.
3. Calculating the integral term $K_i \int_0^t e_v(\tau) d\tau$ using numerical integration techniques.
4. Summing the proportional and integral terms to generate the control input $u(t)$.

4.6 Discrete-Time Implementation

For digital systems, the PI controller is implemented in discrete time. Using a sampling time Δt , the integral term is approximated as:

$$\int_0^t e_v(\tau) d\tau \approx \sum_{k=0}^n e_v(k\Delta t) \Delta t,$$

and the control law becomes:

$$u[n] = K_p e_v[n] + K_i \Delta t \sum_{k=0}^n e_v[k],$$

where $u[n]$ is the control input at the n -th time step.

4.7 Tuning the Controller

The performance of the PI controller depends on the choice of K_p and K_i . These parameters are tuned based on:

- Stability: Ensuring the system remains stable under feedback.
- Response speed: Achieving desired velocity quickly.
- Steady-state error: Minimizing residual errors over time.

4.8 Application in Longitudinal Control

In the Automated Parking Valet system, the PI controller determines the acceleration a_r or deceleration needed to follow the desired speed profile. The control input $u(t)$ directly affects the vehicle's longitudinal dynamics:

$$\dot{v}_r = a_r = u(t),$$

where $u(t)$ is computed using the PI control law.

4.9 Performance Evaluation

The effectiveness of the PI controller is measured by analyzing:

- Speed tracking accuracy: The ability to follow the desired speed profile.
- Stability under disturbances: Robustness to noise and variations in load or terrain.

A well-tuned PI controller provides smooth acceleration and deceleration, ensuring passenger comfort and system reliability.

5 Simulation Setup

5.1 Simulator Design

A vehicle simulator executes control commands using the bicycle model. The vehicle's trajectory is monitored, and errors in path tracking are corrected dynamically.

5.2 Path Planning

Hybrid A* and RRT planners generate collision-free paths considering:

- Obstacle locations.
- Vehicle dynamics and dimensions.
- Smooth transitions for forward and reverse motions.

6 Results and Analysis

6.1 Trajectory Tracking

The system successfully tracked the smoothed reference trajectory under various conditions. A combination of lateral and longitudinal controls ensured precise maneuvers.

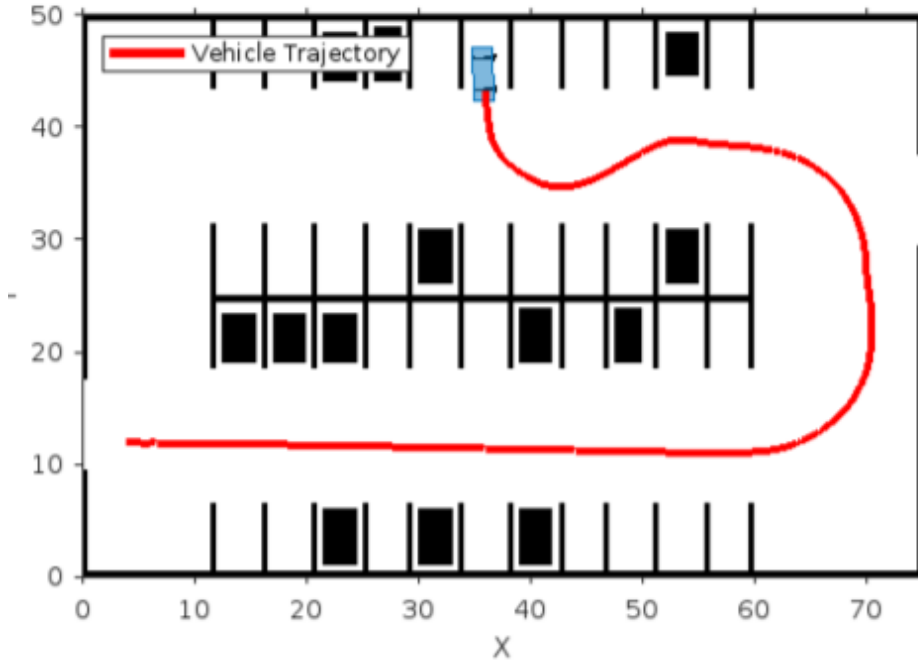


Figure 1: Simulated Trajectory Tracking Results

6.2 Parking Maneuvers

Forward and reverse parking maneuvers were validated in simulations, showcasing efficient handling of tight spaces.

6.3 Error Analysis

The controller effectively mitigated errors from:

- Tire slippage.
- Localization inaccuracies.

7 Related Research

7.1 Influential Papers

1. Buehler et al., *The DARPA Urban Challenge* [1]: This paper provided insights into trajectory planning and control for autonomous vehicles in urban environments. The concepts of hybrid path planning were adapted for the parking system.
2. Lepetic et al., *Time Optimal Path Planning* [2]: The velocity profile generation and acceleration constraints in this paper influenced the longitudinal control design.

7.2 Application of Research

The research papers guided:

- Selection of kinematic models for low-speed scenarios.
- Development of smooth trajectory generation techniques.
- Implementation of proportional-integral-based longitudinal control.

8 Conclusion and Future Work

The Automated Parking Valet system demonstrated successful parking maneuvers using a robust control framework. Future enhancements could include:

- Integration of dynamic maps with V2X communication.
- Adoption of machine learning for enhanced obstacle detection.
- Optimization for real-time implementation in hardware systems.

A Appendix

Detailed mathematical derivations and auxiliary functions are available in the supplementary materials.

References

- [1] Buehler, M., Iagnemma, K., & Singh, S. *The DARPA Urban Challenge: Autonomous Vehicles in City Traffic*. Springer, 2009.
- [2] Lepetic, M., et al. Time Optimal Path Planning Considering Acceleration Limits. *Robotics and Autonomous Systems*, 2003.